MANICODE SECURITY

PROMPT PLAYBOOK

EDITION 05

A SPEC-DRIVEN WORKFLOW FOR CLAUDE CODE

BOOTSTRAP AN EMPTY REPO

Twelve prompts. A complete specification. A clear path to code.

Build requirements, design, architecture and security rules; create a reusable requirement template; threat-model the specifications; resolve decisions; then turn the GitHub backlog into implementation, one pull request at a time.

RUN THE PROMPTS IN ORDER

STEPS 01 - 12 / REQUIREMENTS TO CODE

JIM MANICO

MANICODE SECURITY

TWELVE STEPS, IN ORDER

Paste one complete step into Claude Code at a time. Prompts that span several pages must be copied together. Replace the project inputs and local paths before use.

01	**CREATE REQUIREMENTS** REQUIREMENTS.md	03
02	**CREATE DESIGN & LOGO** DESIGN.md + logo.svg	05
03	**DEFINE ARCHITECTURE** ARCHITECTURE.md	06
04	**BUILD SECURITY RULES** SECURITY.md	08
05	**CREATE REQUIREMENT TEMPLATE** REQUIREMENT_TEMPLATE.md	11
06	**CREATE AGENT INSTRUCTIONS** AGENTS.md	16
07	**RECONCILE SPECIFICATION OWNERSHIP** Five specification files	17
08	**THREAT-MODEL THE SPECIFICATIONS** THREAT_MODEL.md + affected specifications	18
09	**RESOLVE OPEN DECISIONS** Specifications / first question pass	20
10	**CREATE THE GITHUB BACKLOG** Epic + implementation sub-issues	21
11	**RECONCILE THE BACKLOG** Specifications + issues / second question pass	24
12	**IMPLEMENT IN DEPENDENCY ORDER** One branch + one PR per leaf issue	26

Source: SPEC-DRIVEN-BOOTSTRAP.md
Full prompt text is retained, with typographic dashes rendered as standard hyphens. The original Markdown is also embedded as a PDF attachment.

01

PROMPT

CREATE REQUIREMENTS

REQUIREMENTS.md COPY THE COMPLETE STEP

```
Create `REQUIREMENTS.md` in this repository. `REQUIREMENTS.md` should own WHAT the
system must do: functional and non-functional requirements, each independently
testable.

The project notes below are the sole source of truth. The repository is empty. Do not
infer behavior. Produce the best document the notes support and record unresolved
decisions in `## Open Questions`. Please use ASD-STE100 for these requirement
descriptions.

# Project Notes

"<<PROJECT_NOTES>>"

# Output

Save `REQUIREMENTS.md`, then return only its complete Markdown contents. It MUST
contain exactly these `##` sections, in this order:

## Required Requirement Inputs

Include each field exactly once, filled only from the project notes. Use `UNKNOWN`
when not provided. Never invent details.

- Project purpose:
- Primary users / actors:
- Core workflows:
- Business objects / data entities:
- External integrations:
- Authentication / roles:
- Regulatory or privacy constraints:

## Functional Requirements

Convert the notes into concise functional requirements:

- Group under `###` headings by capability or workflow.
```

STEP 01 CONTINUES ON THE NEXT PAGE

01

PROMPT / CONTINUED

CREATE REQUIREMENTS

REQUIREMENTS.md COPY THE COMPLETE STEP

```
- Use stable IDs and this format: `- **FR-1.1** The system MUST ...`
- Use normative terms where appropriate: MUST, MUST NOT, SHOULD, MAY.
- One independently testable, externally observable behavior per requirement. No
implementation details.
- Identify actor, action, system response, and data affected when known.
- Include validation, authorization, and failure behavior when stated or clearly
required by the workflow.
- If a missing detail makes a requirement untestable, move the issue to `## Open
Questions`.

## Non-Functional Requirements

Include only non-functional requirements the notes state, such as performance,
availability, or capacity. Apply the functional-requirement rules above with `NFR-`
IDs: `- **NFR-1.1** The system MUST ...`. If the notes state none, write `None
stated`.

## Open Questions

List only unresolved questions that materially affect system behavior, scope, or
requirement testing, and that the notes do not already answer.

- **OQ-1** ...
- **OQ-2** ...

Keep the document compact, concrete, and optimized as context for spec-driven
development.
```

02

PROMPT

CREATE DESIGN & LOGO

DESIGN.md + logo.svg

COPY THE COMPLETE STEP

```
Create `DESIGN.md`, then `logo.svg`, in the repository root.

Read `REQUIREMENTS.md` before writing. It is the sole source of truth for behavior and
workflows. Assume no implementation or code exists.

Marker Convention: Use `UNKNOWN` for facts not provided; use `TO BE DECIDED` for
decisions not yet made; label unsupported inferences as `ASSUMPTION`. Never invent
values.

# logo.svg
Generate a simple, clean, valid SVG logo that MUST:
- Be a single primary mark, recognizable at both small and large sizes.
- Use a highly restrained, deliberate color palette matching the DESIGN.md tokens.
- Be plain SVG with a `viewBox` and a `<title>`, and no external assets, scripts,
event-handler attributes, or `<foreignObject>`; prefer vector shapes/paths over text
tags.

# DESIGN.md Implementation Guardrails
- Strictly comply with the official @google/design.md spec (YAML front matter +
required prose sections).
- Every foreground/background token pair the design uses MUST meet WCAG 2.2 AA
contrast: 4.5:1 for normal text, 3:1 for large text, and 3:1 for UI components, focus
indicators, and graphical objects required to understand content.
- Record unresolved design decisions under a `## Open Questions` section as `DQ-1`,
`DQ-2`, ...; the `lint` check above must still pass.
- Provide implementation-neutral, compact component patterns without choosing a CSS
framework.

# Constraints
Strictly no UI/CSS framework choices, directory maps, code snippets, deployment
topology, speculative features, or detailed security controls. Keep all files compact,
concrete, and optimized for AI context loading.
```

PROMPT

03 DEFINE ARCHITECTURE

ARCHITECTURE.md COPY THE COMPLETE STEP

```
Create `ARCHITECTURE.md` in the repository root. It owns HOW the system is built:
logical components, boundaries, technology choices, and data flow.

Read `REQUIREMENTS.md` for system behavior and `DESIGN.md` for visual, interaction,
and frontend constraints. Assume no implementation exists. If either file is missing,
report it and stop.

## Developer Inputs

- Server framework: <<SERVER_FRAMEWORK_OR_NONE>>
- Server runtime: <<SERVER_RUNTIME_OR_NONE>>
- Client framework(s): <<CLIENT_FRAMEWORKS_OR_NONE>>
- Architecture notes: <<ARCHITECTURE_NOTES_OR_NONE>>

Ask only for missing server or client framework inputs required by confirmed target
surfaces. Treat `NONE` as an intentional choice. Record stated versions exactly;
`SECURITY.md` selects packs only for confirmed versions.

## Rules

- Honor explicit requirements and technology choices. Flag conflicts without silently
resolving them.
- Recommend the simplest architecture that satisfies the requirements. Label
unconfirmed technical choices `PROPOSED` and briefly explain consequential choices.
- Mark missing facts `UNKNOWN` and unmade decisions `TO BE DECIDED`. Never invent
features, integrations, or scale targets.
- Record all other unresolved architecture choices under `## Open Questions` for the
later question phase; do not ask them during architecture generation.
```

STEP 03 CONTINUES ON THE NEXT PAGE

03

PROMPT / CONTINUED

DEFINE ARCHITECTURE

ARCHITECTURE.md COPY THE COMPLETE STEP

```
## Document Structure

1. **Required Architecture Inputs:** A `## Required Architecture Inputs` section
listing each field once: requirements source (`REQUIREMENTS.md`), design source
(`DESIGN.md`), target surfaces (derived from `REQUIREMENTS.md`; `UNKNOWN` if
unresolved), runtime and deployment environment (confirmed inputs or `UNKNOWN`),
server framework, client framework(s), API style and integration model, authentication
and session model, data model and scale expectations, and security expectations
(deferred to `SECURITY.md`). Fill fields only from developer inputs, architecture
notes, or requirements; otherwise use `PROPOSED` for a recommendation or `TO BE
DECIDED` for an identified pending decision.
2. **Architecture and decisions:** State the frameworks, overall architecture, and
necessary choices for APIs, persistence, and authentication/session handling.
3. **Components and flows:** Use a compact table covering responsibilities,
business-rule enforcement, data ownership, and exact requirement IDs where available,
each marked `SUPPORTED`, `PARTIALLY DEFINED`, or `TO BE DECIDED`. Describe the main
data flows and trust boundaries.
4. **Dependency rules and open decisions:** Define permitted dependency directions as
stable `DR-1`, `DR-2`, ... rules ensuring explicit data ownership, unidirectional
flow, and no circular dependencies. Keep business rules separate from presentation and
vendor-specific code. List unresolved issues that materially affect the architecture
under `## Open Questions` as `AQ-1`, `AQ-2`, ....
5. **Diagrams:** Include only Mermaid diagrams that materially clarify the
architecture: one component and trust-boundary diagram, plus diagrams for main data
flows when useful. Include only elements supported by the requirements.

Keep the document concise and concrete. Include only components justified by
requirements. Exclude code, directory trees, class inventories, boilerplate, and
deployment topology. Preserve security requirements and their enforcement boundaries;
defer detailed controls to `SECURITY.md`.
```

04

PROMPT

BUILD SECURITY RULES

SECURITY.md

COPY THE COMPLETE STEP

```
# Build `SECURITY.md`

Create or replace only the repository-root `SECURITY.md`. Write secure-coding
implementation rules only. Do not perform threat modeling, score risks, write
application code, or modify other files.

## Authority and inputs

Read `REQUIREMENTS.md`, `DESIGN.md`, and `ARCHITECTURE.md`, then the security notes
below and any existing `SECURITY.md`.

- `REQUIREMENTS.md` owns behavior, users, data, integrations, and compliance
obligations.
- `DESIGN.md` owns user interaction, rendering, and browser behavior.
- `ARCHITECTURE.md` owns components, interfaces, data flows, and trust boundaries.

Security notes and existing rules may suggest controls or preserve IDs; they cannot
establish project facts or override these documents. Record conflicts without choosing
a resolution.

Do not ask questions or invent architecture, technologies, versions, providers,
mechanisms, obligations, or configuration values. Mark missing facts `UNKNOWN` and
unresolved choices or conflicts `TO BE DECIDED`. Neither establishes a pack's or
rule's applicability.

### Security notes

<<SECURITY_NOTES_OR_NONE>>

## Manicode candidates

Corpus root: `/Users/jmanico/Dropbox/github/aegis/secure-coding-knowledge`
```

STEP 04 CONTINUES ON THE NEXT PAGE

04

PROMPT / CONTINUED

BUILD SECURITY RULES

SECURITY.md

COPY THE COMPLETE STEP

```
Evaluate these candidate packs for `SECURITY.md`:

- Server runtime: `js/nodejs/26`
- Client framework: `js/react-native/0.87`
- Server framework: `js/fastify/5.12`
- Application AuthN: `js/better-auth/1.7`
- Identity provider / WebAuthn: `jvm/keycloak/26.7`
- Android passkeys: `jvm/android-sdk/37`
- iOS passkeys: `swift/ios-sdk/27`
- Application AuthZ: `policy/cedar/4.12`
- Relationship AuthZ: `policy/openfga/1.19`
- Relationship AuthZ: `policy/spicedb/1.56`
- Gateway/general policy: `policy/open-policy-agent/1.20`
- Validation: `js/zod/4`
- Data access: `js/prisma/7.10`
- Containers: `infra/dockerfile/1`

Mark a direct pack `SELECTED` only when authoritative project documents confirm the
technology and applicable version scope. Record unresolved selections as open
decisions.

Many of these packs reference parent packs. Evaluate those transitively as well.

## Required `SECURITY.md` structure

Use these four `##` sections, in order, with compact Markdown entries rather than wide
tables.

## Secure-Coding Inputs

List documents read or missing. Summarize only confirmed project facts needed to
justify selections and rules. Identify material gaps and conflicts.

## Selected Manicode Prompts

Record the corpus root used for resolution. List direct selection evidence, then the
ordered composition, distinguishing direct and transitive packs. Record prompt and
source paths relative to the stated absolute corpus root; list the preamble
separately.
```

STEP 04 CONTINUES ON THE NEXT PAGE

04

PROMPT / CONTINUED

BUILD SECURITY RULES

SECURITY.md COPY THE COMPLETE STEP

```
## Secure-Coding Rules

Preserve existing `SC-###` IDs where their meaning remains unchanged. Use:

**SC-001 — Short name**
- **Status:** ACTIVE
- **Rule:** A concise, project-specific `MUST` or `MUST NOT`.
- **Enforcement:** Confirmed component or implementation point.
- **Evidence:** Precise project-document reference and eligible resolved
prompt/preamble reference; verified supporting documentation where used.
- **Verification:** Observable pass/fail check.

Every active rule must have all five fields. Name exact APIs only when their
technology and version applicability are confirmed. Do not invent limits, algorithms,
retention periods, assurance levels, or implementation mechanisms. Do not copy prompts
wholesale, include secrets, or provide exploit instructions.

When any required fact, decision, source, or applicability check is unresolved, use a
**BLOCKED** entry naming the dependency and linked `OD-###` decision. Blocked entries
are not implementation instructions; do not fill their gaps with speculative rules.

## Open Security Decisions

For each stable `OD-###` entry, record:

- Missing fact, unresolved choice, or conflict and the evidence needed to resolve it.
- Affected rule IDs or coverage area.
- Candidate packs, if useful.
- Owner only if documented; otherwise `UNKNOWN`.

Write `None` only when nothing remains unresolved.

Before saving, check authority, composition eligibility, provenance, applicability,
and testability. Keep unresolved items blocked. Save only `SECURITY.md` and return its
complete Markdown contents.
```

PROMPT

05 CREATE REQUIREMENT TEMPLATE

REQUIREMENT_TEMPLATE.md COPY THE COMPLETE STEP

````
Create or replace `REQUIREMENT_TEMPLATE.md` in the repository root.

This is a literal file-generation task. Do not tailor the template to the current repository, and do not fill any placeholder from `REQUIREMENTS.md`, `DESIGN.md`, `ARCHITECTURE.md`, or `SECURITY.md`.

Write exactly the Markdown contained between the `<FILE_CONTENT>` and `</FILE_CONTENT>` markers.

Do not include the markers in the file. Do not include a surrounding code fence. Do not replace, complete, or remove bracketed placeholders. Do not add introductory text, explanations, examples, or additional sections.

<FILE_CONTENT>

# Requirement Template — Web, Mobile & API Applications

Use one copy of this template for each atomic requirement.

Replace bracketed placeholders when authoring a requirement. Use `N/A` only when a field has been evaluated and is not applicable. Use `TO BE DECIDED` when the answer is unresolved.

Uppercase normative terms use BCP 14 semantics as defined by RFC 2119 and RFC 8174. Prefer `MUST`, `MUST NOT`, `SHOULD`, `SHOULD NOT`, and `MAY`. Split compound behavior into separate requirements.

## Metadata

- **ID**: [REQ-MODULE-###, e.g., REQ-AUTH-001]
- **Title**: [Concise requirement name]
- **Version**: [Semantic version, e.g., 1.0.0]
- **Status**: [Draft | In Review | Approved | Implemented | Verified | Deprecated]
- **Owner**: [Person or team responsible for the requirement]
- **Author**: [Name]
- **Last Updated**: [ISO 8601 date: YYYY-MM-DD]
- **Priority**: [Critical | High | Medium | Low]
- **Requirement Type**: [Functional | Security | Privacy | Compliance | Performance | Reliability | Operational]
- **Source / Parent**: [Source requirement IDs, stakeholder decision, threat-model decision, legal obligation, issue, or other origin]
````

STEP 05 CONTINUES ON THE NEXT PAGE

05

PROMPT / CONTINUED

CREATE REQUIREMENT TEMPLATE

REQUIREMENT_TEMPLATE.md COPY THE COMPLETE STEP

```
## Requirement

- **Statement**: [One atomic, unambiguous, and independently testable statement describing what the system MUST, MUST NOT, SHOULD, SHOULD NOT, or MAY do.]
- **Rationale**: [Business, user, regulatory, architectural, privacy, or threat-driven reason for the requirement. Do not introduce additional required behavior here.]
- **Assumptions**: [Assumptions on which the requirement depends, or `None`.]
- **Out of Scope**: [Behavior that is explicitly excluded from this requirement, or `None`.]
- **Design Traceability**: [Relevant `DESIGN.md` section or decision, or `N/A`. Do not invent missing design decisions.]
- **Architecture Traceability**: [Relevant `ARCHITECTURE.md` component, boundary, data flow, or dependency rule.]
- **Security Traceability**: [Relevant `SECURITY.md` rule IDs, or `N/A`.]

## Scope

- **Applies To**: [Web Client | Mobile Client | Server-Side Application | API | Background Processing | External Integration | Multiple]
- **Components**: [Affected logical components using the names defined in `ARCHITECTURE.md`]
- **Interfaces / Operations**: [Affected pages, screens, workflows, API operations, events, or commands]
- **Actors**: [Affected users, roles, service identities, administrators, or external systems]
- **Preconditions**: [State that MUST exist before the behavior occurs, or `None`]
- **Data Classification**: [Public | Internal | Confidential | Restricted | Multiple | N/A]
- **Personal or Regulated Data**: [None | Personal Data | Sensitive Personal Data | Payment Account Data | Health Data | Financial Data | Other]
- **Jurisdiction / Regulatory Scope**: [Established jurisdiction or regulatory scope, `N/A`, or `TO BE DECIDED`]
```

STEP 05 CONTINUES ON THE NEXT PAGE

05

PROMPT / CONTINUED

CREATE REQUIREMENT TEMPLATE

REQUIREMENT_TEMPLATE.md COPY THE COMPLETE STEP

```
## Security Context

- **Security Objectives**: [Confidentiality | Integrity | Availability | Authenticity
| Authorization | Accountability | Privacy | Safety | Multiple | N/A]
- **Control Layers**: [Architecture | Authentication | Authorization | Session
Management | Input Validation | Business-Rule Validation | Output Encoding |
Sanitization | Data Protection | Logging and Monitoring | Availability | Supply Chain
| Other]
- **Threat References**: [Versioned threat references such as CWE, CAPEC, STRIDE, or
an OWASP Top 10:2025 category; use `N/A` when no threat mapping applies]
- **Abuse / Misuse Case**: [What an attacker, unauthorized actor, or compromised
dependency may attempt, or `N/A`]
- **Trust Boundary**: [Boundary at which untrusted data, identity assertions,
commands, or responses enter a more trusted component]
- **Untrusted Inputs or Assertions**: [Inputs, identity claims, authorization context,
external responses, files, events, or other data that MUST NOT be trusted without
verification]
- **Authoritative Enforcement Point**: [Server-side component or other trusted
boundary responsible for enforcing the requirement]
- **Independent Verification**: [Validation or authorization performed independently
of the originating client, actor, or external system]
- **Zero Trust Relevance**: [Specific NIST SP 800-207 resource-access principle or
section, or `N/A`. Do not use Zero Trust as a synonym for input validation.]

## Standards Alignment

Only include mappings verified against the cited version of the standard. Do not guess
control identifiers.

Use `N/A` when a standard has been evaluated and does not apply. Use `TO BE DECIDED`
when applicability or mapping has not yet been assessed.

- **OWASP ASVS 5.0.0**: [Versioned requirement ID, e.g., `v5.0.0-1.2.5`, or `N/A`]
- **OWASP AISVS 1.0**: [Requirement ID when an AI-enabled component is involved, or
`N/A`]
- **NIST SP 800-53 Rev. 5**: [Control or control-enhancement ID and the catalog
release used, or `N/A`]
- **NIST SP 800-207**: [Specific Zero Trust tenet or section when applicable, or
`N/A`]
- **Regulatory**: [Applicable regulation, exact version or effective date, and
specific requirement or section; otherwise `N/A` or `TO BE DECIDED`]
- **Other**: [Versioned RFC, ISO/IEC standard, industry standard, or organizational
policy and exact section, or `N/A`]
- **Mapping Basis**: [Brief explanation of why each listed mapping applies]
```

STEP 05 CONTINUES ON THE NEXT PAGE

05

PROMPT / CONTINUED

CREATE REQUIREMENT TEMPLATE

REQUIREMENT_TEMPLATE.md COPY THE COMPLETE STEP

```
## Acceptance Criteria

Each acceptance criterion MUST:

- describe one observable pass/fail outcome
- be independently testable
- remain within the behavior defined by the requirement
- identify relevant preconditions, action, and expected result
- avoid introducing new product behavior

Use Given/When/Then or an equally explicit pass/fail form.

1. **AC-01 - Expected behavior**: Given [valid preconditions], when [actor or system
action], then [observable expected result].
2. **AC-02 - Boundary or failure behavior**: Given [boundary condition, invalid input,
unauthorized action, or dependency failure], when [action occurs], then [safe and
observable result with no unauthorized state change or sensitive side effect].
3. **AC-03 - Prohibited behavior**: Given [relevant preconditions], when [action
occurs], then [behavior or outcome that MUST NOT occur].
4. **AC-04 - Additional criterion**: [Add only when needed. Remove when not
applicable.]

## Failure Behavior

- **On Invalid Input**: [Protocol-appropriate rejection behavior, error
classification, and confirmation that no protected state change or sensitive side
effect occurs]
- **On Authentication Failure**: [Expected response and disclosure behavior, or `N/A`]
- **On Authorization Failure**: [Deny the operation; define response behavior and
whether resource existence may be disclosed, or `N/A`]
- **On Security-Decision Failure**: [Deny by default unless an explicit, documented
risk decision permits another behavior]
- **On External Dependency Failure**: [Timeout, retry, circuit-breaker, idempotency,
or degradation behavior, or `N/A`]
- **On System Error**: [Rollback, consistency, recovery, and safe error-response
behavior; internal state and sensitive data MUST NOT be disclosed]
- **Logging / Audit**: [Events and fields to record, correlation mechanism, redaction
requirements, and prohibited data]
- **Alerting**: [Alert condition, threshold, severity, and destination, or `N/A`]
```

STEP 05 CONTINUES ON THE NEXT PAGE

05

PROMPT / CONTINUED

CREATE REQUIREMENT TEMPLATE

REQUIREMENT_TEMPLATE.md COPY THE COMPLETE STEP

```
## Test Strategy

- **Unit Tests**: [Business rules, validation rules, authorization decisions,
transformations, and error paths testable in isolation]
- **Integration Tests**: [Cross-component, persistence, identity, external
integration, or trust-boundary behavior]
- **Security Tests**: [Applicable negative tests, abuse cases, authorization tests,
fuzzing classes, SAST rules, DAST targets, or manual review items]
- **Compliance Tests / Evidence**: [Automated or manual evidence required for an
applicable obligation, or `N/A`]
- **Acceptance-Criteria Traceability**: [Test identifier or test suite covering each
`AC-*` criterion]
- **Coverage Target**: [Project-defined target. All security-critical decisions and
error paths MUST include positive and negative test coverage.]
- **Required Test Environment**: [Fixtures, identities, roles, dependency simulators,
configuration, or test data required]

## Dependencies

- **Upstream Requirements**: [Requirement IDs that MUST be satisfied first, or `None`]
- **Downstream Requirements**: [Requirement IDs that depend on this requirement, or
`None`]
- **External Dependencies**: [Third-party services, identity providers, APIs,
cryptographic services, hardware services, or `None`]
- **Dependency Assumptions**: [Security, availability, data-integrity, or contractual
assumptions made about external dependencies]
- **Failure Impact**: [Effect of an unavailable, compromised, slow, or malformed
dependency response]

## Implementation Notes

- **Constraints**: [Mandatory technology, compatibility, latency, availability,
migration, legal, or deadline constraints]
- **Prohibited Approaches**: [Explicit anti-patterns, such as relying solely on
client-side enforcement or logging credentials, tokens, or sensitive raw data]
- **Implementation Guidance**: [Non-normative guidance that does not add or alter
required behavior]
- **AI Development Guidance**: [Applicable prompt-library files, repository
instructions, secure-coding rules, and mandatory human-review gates, or `N/A`]
- **Required Human Review**: [Security, privacy, architecture, legal, product, or
domain reviewers required before approval or release]
- **Open Decisions**: [Unresolved decisions that prevent approval or implementation,
or `None`]

</FILE_CONTENT>
```

06

PROMPT

CREATE AGENT INSTRUCTIONS

AGENTS.md COPY THE COMPLETE STEP

```
Create `AGENTS.md` in the repository root.

The repository is am application that has been specified but not built yet. The
specification files are `REQUIREMENTS.md`, `ARCHITECTURE.md`, `SECURITY.md`,
`DESIGN.md`, and `REQUIREMENT_TEMPLATE.md`. Read all five before writing anything. Do
not invent workflow, tooling, or process details not defined in those documents; mark
undecided workflow items with explicit `TO BE DECIDED` placeholders.

## `AGENTS.md`

Focused on project behavior, guardrails, and working style:

- Import the four spec files with `@REQUIREMENTS.md`, `@ARCHITECTURE.md`,
`@SECURITY.md`, and `@DESIGN.md`. If any of the four is missing, stop and report the
error instead of generating the file.
- Do not restate rules the imported files already cover; `AGENTS.md` adds only what
they don't. Every rule appears exactly once.
- Include this rule: every leaf implementation issue MUST follow
`REQUIREMENT_TEMPLATE.md`; epic, workstream, and decision issues use concise
purpose-specific bodies that link to affected specification IDs and issues.
- No fluff, low-signal content, unnecessary rules, or historical/meta commentary. Make
it compact, concise and meaningful.
```

07

PROMPT

RECONCILE SPECIFICATION OWNERSHIP

Five specification files

COPY THE COMPLETE STEP

```
Review the five specification files below so each owns exactly one domain, nothing is
duplicated across files, and no information is lost. Move every rule to the file that
owns its domain. Do not modify `REQUIREMENT_TEMPLATE.md` or `logo.svg`.

## Ownership

- `REQUIREMENTS.md` - WHAT the system must do: functional and non-functional
requirements, each independently testable, including externally observable validation,
authorization, and failure behavior.
- `DESIGN.md` - the visual and frontend design language: brand and logo, color
palette, typography, layout, component conventions, accessibility targets, and
security-relevant user interactions and rendering behavior.
- `ARCHITECTURE.md` - HOW it is built: components, interfaces, data flow, trust
boundaries, technology choices and rationale.
- `SECURITY.md` - the security posture: secure-coding rules, controls, trust-boundary
enforcement, and open security decisions. All security controls are authored here.
- `AGENTS.md` - agent operating rules: coding conventions, enforcement rules,
build/test/run commands, do/don't guidance. It defines no system behavior or
architecture.

Non-owning files reference the owning file; they never restate its content (no
repeated design tokens, security requirements, or architecture decisions). Tiebreak
when content fits more than one domain: security controls go to `SECURITY.md`;
externally observable product behavior stays in `REQUIREMENTS.md`; everything else
goes to the owning file above.

## Procedure

1. **Inventory** - list every distinct requirement, rule, claim, and decision entry
(`FR-*`, `NFR-*`, `OQ-*`, `DQ-*`, `AQ-*`, `DR-*`, `SC-*`, `OD-*`) across all five
files and tag each with its owning file.
2. **Detect** - flag (a) duplicates, (b) contradictions, (c) low-signal content:
filler, motivational text, restated-obvious points, dead links.
3. **Resolve** - for each duplicate, keep the copy in the owning file and replace the
rest with at most a one-line cross-reference. Delete low-signal content; when unsure,
keep it and list it in your final summary. Resolve contradictions using the owning
specification. Correct a conflicting derivative statement when ownership is
unambiguous. Otherwise, preserve the alternatives in the owning specification under
its stable open-item ID (`OQ-*`, `DQ-*`, `AQ-*`, or `OD-*`). Do not choose an
interpretation merely because it appears more secure. Other artifacts reference the
decision ID and mirror its status; IDs are never reused.
4. **Verify** - re-read all five files end to end. Confirm no requirement, control,
constraint, or threat mapping was dropped, every cross-reference and ID reference
resolves to a real entry, and each file stands on its own.
```

08

PROMPT

THREAT-MODEL THE SPECIFICATIONS

THREAT_MODEL.md + affected specifications

COPY THE COMPLETE STEP

```
# Threat-model and harden the specifications

Before implementation, run the complete threat-modeling pipeline and update the
specifications with its supported findings. Do not ask questions or analyze imagined
code.

## Inputs

Read:

1. `REQUIREMENTS.md`
2. `ARCHITECTURE.md`
3. `SECURITY.md`
4. `DESIGN.md`

Missing information is an evidence gap, not proof of a vulnerability. Do not invent
architecture, controls, risk scores, technologies, owners, or deadlines.

## Run every threat-modeling agent

Pipeline:

`/Users/jmanico/Dropbox/github/aegis/security-utilities-knowledge/content/pipelines/threat-modeling`

Read `README.md` and run every numbered agent:

1. Run stages `00`–`07` independently, in parallel where possible, using all supplied
specifications as evidence.
2. Give every resulting fragment to `08-consolidator.md`.
3. Run `09-diagram-generator.md`, then `10-visual-design-converter.md`.
4. Run `11-report-generator.md` on the most downstream artifact.
```

STEP 08 CONTINUES ON THE NEXT PAGE

08

PROMPT / CONTINUED

THREAT-MODEL THE SPECIFICATIONS

THREAT_MODEL.md + affected specifications COPY THE COMPLETE STEP

```
Run every stage even when its method is not applicable; record `Not applicable` or the
evidence gap instead of inventing findings. Stage `06` must record that no
implementation exists.

Save the final stage-11 report as `THREAT_MODEL.md`, preserving the unified model,
stable `T-nnn` IDs, evidence, conflicts, gaps, diagrams, and coverage.

## Update the specifications

Apply evidence-supported changes:

- `SECURITY.md`: secure-coding rules and blocked security decisions.
- `REQUIREMENTS.md`: required product behavior.
- `ARCHITECTURE.md`: components, trust boundaries, data flows, enforcement placement,
and affected diagrams.
- `DESIGN.md`: security-relevant user interactions and rendering behavior.
- `THREAT_MODEL.md`: threats, risks, evidence, gaps, and coverage.

Use each file's existing format and IDs. Link every change to its driving `T-nnn` ID.
Put each decision in one owning file; use references instead of duplication.

If a mitigation requires an unresolved decision, add an `OD-###` entry to
`SECURITY.md` and keep the affected `SC-###` rule `BLOCKED`.

Do not weaken existing requirements, treat planned controls as implemented, or add
exploit instructions.
```

09

PROMPT

RESOLVE OPEN DECISIONS

Specifications / first question pass

COPY THE COMPLETE STEP

```
Scan the entire repo for unresolved items. Skip `REQUIREMENT_TEMPLATE.md`; its
placeholders are intentional. Look for:

- any "## Open Questions" section (REQUIREMENTS.md, DESIGN.md, ARCHITECTURE.md)
- every `TO BE DECIDED`, `UNKNOWN`, `TBD`, `PROPOSED`, `DEFERRED`, or unresolved
`<<PLACEHOLDER>>` or `{{PLACEHOLDER}}` marker
- "## Required Requirement Inputs", "## Required Architecture Inputs", and "##
Secure-Coding Inputs" lines that are still blank or provisional
- assumptions explicitly labeled provisional or "assumption" in DESIGN.md,
ARCHITECTURE.md, and SECURITY.md
- every `BLOCKED` rule and open `OD-###` entry in SECURITY.md
- threat model findings in THREAT_MODEL.md that still need a decision

Collect all of these into a single ordered list, most foundational first (requirements
and architecture before downstream details).

Then ask me the questions ONE AT A TIME as if I was 10 years old. Wait for my answer
to each before asking the next. Do not batch them. Do not guess, and do not fill
anything in on my behalf. If my answer raises a new question, add it to the list.

After each answer, record it. Once every question is answered:

- update REQUIREMENTS.md, DESIGN.md, ARCHITECTURE.md, SECURITY.md, and THREAT_MODEL.md
with the resolved values
- remove the `TO BE DECIDED` / `UNKNOWN` markers and resolved placeholders you
addressed
- clear the "Open Questions" entries and `OD-###` decisions you resolved, and turn a
`BLOCKED` rule into a complete `ACTIVE` rule only when all of its dependencies are
resolved
- keep anything I deferred clearly visible so it is not lost

Ensure every pending question is answered or explicitly marked `DEFERRED` under its
decision ID, assigning the owning file's open-item ID first if it has none. Deferred
decisions remain visible and block only dependent work.

After resolving decisions, update their owning specifications and refresh all affected
downstream artifacts, including secure-coding rules, threat findings, acceptance
criteria, coverage mappings, and dependencies.
```

PROMPT

10 CREATE THE GITHUB BACKLOG

Epic + implementation sub-issues COPY THE COMPLETE STEP

```
# Create the GitHub implementation backlog

Turn the complete repository specification into one GitHub epic and
implementation-ready sub-issues. Create the issues; do not stop for questions or leave
work untracked.

## Sources

Read:

- `REQUIREMENTS.md`
- `ARCHITECTURE.md`
- `SECURITY.md`
- `DESIGN.md`
- `THREAT_MODEL.md`, if present
- `REQUIREMENT_TEMPLATE.md`
- `AGENTS.md`, for repository instructions only

Each specification owns its domain. Never resolve conflicts by reading order or invent
missing scope.

## Plan

Map every requirement, security rule, threat, architecture component, and design
constraint into:

1. One root epic
2. Optional workstream issues
3. Decision issues for unresolved or conflicting specifications
4. Leaf implementation issues

Create a coverage matrix and acyclic dependency graph before touching GitHub. Put the
coverage matrix in the epic, continuing in epic comments if it exceeds GitHub's issue
body limit.
```

STEP 10 CONTINUES ON THE NEXT PAGE

10

PROMPT / CONTINUED

CREATE THE GITHUB BACKLOG

Epic + implementation sub-issues COPY THE COMPLETE STEP

```
Nothing may be omitted:

- Every requirement maps to an issue.
- Every applicable `SC-*` rule and `T-*` threat maps to implementing issues.
- Every unresolved decision becomes a prerequisite `REQ-DECISION-###` issue.
- Dependent implementation issues link to that decision and remain blocked until it
closes.

## Issue quality

Every leaf issue must:

- Implement one coherent behavior, control, endpoint, workflow, or data flow.
- Require no undocumented decisions.
- Estimate up to 2 engineer-days and no more than 1,500 changed human-authored lines.
- Include implementation, tests, errors, security, and accessibility where applicable.
- Provide Given/When/Then criteria for success, invalid input, unauthorized access,
and failure behavior.
- Cite exact requirement, architecture, security, design, and threat references.

Split oversized or independently releasable work. Never create generic “build
backend,” “add security,” or separate cleanup/testing issues.

Use:

- Epic: `REQ-EPIC-001`
- Workstreams: `REQ-<CAPABILITY>-000`
- Decisions: `REQ-DECISION-010` onward
- Leaves: `REQ-<CAPABILITY>-010` onward
- Titles: `[REQ-AUTH-010] Enforce authorization for account retrieval`

Every leaf implementation issue MUST follow `REQUIREMENT_TEMPLATE.md` exactly, with
placeholders removed and `N/A` used only when truly inapplicable. Epic, workstream,
and decision issues use concise purpose-specific bodies linking to affected
specification IDs and issues.

Recommend one verified implementation model per leaf, with a brief issue-specific
rationale. Record the exact model ID or `NOT VERIFIED`.
```

STEP 10 CONTINUES ON THE NEXT PAGE

10

PROMPT / CONTINUED

CREATE THE GITHUB BACKLOG

Epic + implementation sub-issues COPY THE COMPLETE STEP

```
## Local files

GitHub is the system of record. Do not create planning files in the repository or
modify specification files. Write `--body-file` content to a temporary directory
outside the repository. If an issue exists but a later edit, sub-issue link, or
dependency link fails, comment on it starting with `NEEDS INPUT:` and the reason. If
creating an issue fails, add an epic comment starting with `NEEDS INPUT:` with its
planning ID and the reason. Continue with the remaining issues so the final interview
can clean up, but stop if the epic cannot be created or the failure is systemic
(authentication, permissions, rate limiting). Never delete issues, and never report an
action as successful unless you verified it.

## Create the issues

Preflight:

1. Verify `gh auth status`.
2. Confirm the target repository.
3. Inspect existing open and closed issues for planning-ID reuse.
4. Verify the available GitHub CLI or API support for issue creation, editing,
sub-issues, and dependency links.

Reuse an existing issue only when its planning ID and scope match. If an ID belongs to
different scope, allocate the next unused ID and record the change.

Create or reuse issues in topological order:

1. Epic
2. Decision and workstream issues
3. Leaf issues
4. Sub-issue relationships
5. Dependency links and task lists

Use `--body-file` for every creation. Capture and verify every issue number and URL.
Replace local URL placeholders and update GitHub issue bodies with their final links.

If `gh issue create --parent` is unavailable, use the supported GitHub API for
sub-issue relationships.

## Final response

Return:

- Epic URL
- Counts of created, reused, and failed issues, with the planning ID of each failure
```

11

PROMPT

RECONCILE THE BACKLOG

Specifications + issues / second question pass COPY THE COMPLETE STEP

```
Scan the entire repo for unresolved items for the second time. Skip
`REQUIREMENT_TEMPLATE.md`; its placeholders are intentional. Look for:

- any "## Open Questions" section (REQUIREMENTS.md, DESIGN.md, ARCHITECTURE.md, the
epic, sub-issues)
- every `TO BE DECIDED`, `UNKNOWN`, `TBD`, `PROPOSED`, `DEFERRED`, or unresolved
`<<PLACEHOLDER>>` or `{{PLACEHOLDER}}` marker
- "## Required Requirement Inputs", "## Required Architecture Inputs", and "##
Secure-Coding Inputs" lines that are still blank or provisional
- assumptions explicitly labeled provisional or "assumption" in DESIGN.md,
ARCHITECTURE.md, and SECURITY.md
- every `BLOCKED` rule and open `OD-###` entry in SECURITY.md
- threat model findings in THREAT_MODEL.md that still need a decision
- GitHub issues or epic items flagged as blocked, ambiguous, or needing input,
including every open `REQ-DECISION-*` issue
- every `NEEDS INPUT:` comment on the epic or issues

Collect all of these into a single ordered list, most foundational first (requirements
and architecture before downstream details).

Then ask me the questions ONE AT A TIME as if I was 10 years old. Wait for my answer
to each before asking the next. Do not batch them. Do not guess, and do not fill
anything in on my behalf. If my answer raises a new question, add it to the list.
```

STEP 11 CONTINUES ON THE NEXT PAGE

11

PROMPT / CONTINUED

RECONCILE THE BACKLOG

Specifications + issues / second question pass — COPY THE COMPLETE STEP

```
After each answer, record it. Once every question is answered:

- update REQUIREMENTS.md, DESIGN.md, ARCHITECTURE.md, SECURITY.md, THREAT_MODEL.md,
the epic, and any affected sub-issues with the resolved values
- remove the `TO BE DECIDED` / `UNKNOWN` markers and resolved placeholders you
addressed
- clear the "Open Questions" entries and `OD-###` decisions you resolved, and turn a
`BLOCKED` rule into a complete `ACTIVE` rule only when all of its dependencies are
resolved
- after the epic and affected issues are reconciled, close each resolved
`REQ-DECISION-*` issue with a comment recording the answer, so the issues it blocks
become ready
- keep anything I deferred clearly visible so it is not lost

Ensure every pending question is answered or explicitly marked `DEFERRED` under its
decision ID, assigning the owning file's open-item ID first if it has none. Deferred
decisions remain visible and block only dependent work.

After resolving decisions, update their owning specifications and refresh all affected
downstream artifacts, including secure-coding rules, threat findings, acceptance
criteria, coverage mappings, and dependencies. Reconcile the epic and affected issues;
create, split, retire, or re-parent issues where necessary, using the backlog step's
ID and template rules.
```

12

PROMPT

IMPLEMENT IN DEPENDENCY ORDER

One branch + one PR per leaf issue

COPY THE COMPLETE STEP

```
Starting with the main epic, let's code. Implement leaf issues in dependency order,
skipping any still blocked by an open `REQ-DECISION-*` issue. Use one branch and one
PR per issue, linked with `Closes #<number>`. Merge only after the PR passes the
issue's Test Strategy and the checks in `AGENTS.md` and meets every acceptance
criterion. When done, list any skipped issues and why.
```

YOU CAN AND WILL WRITE SECURE CODE.

manicode.com